

DOT E 6635: Artificial Intelligence for Business Research (Spring 2026)

Model-Free Reinforcement Learning

Renyu (Philip) Zhang

1

Core Challenges of RL

- Temporal credit assignment
 - Which earlier actions/decisions deserve "credit" or "blame" for outcomes that happen later?
- Exploration
 - How to efficiently explore to gain information?
- Generalization
 - How policies learned in one environment could generalize?
- RL to solve a large planning problem using a learning approach
 - AlphaGo, AlphaStar, simulated robotics, LLM coding, LLM math, etc.
 - Transition dynamics is known, but with a very very large state space.
 - Run the simulator to collect data.
 - RL proved to be very successful.
- RL to solve a learning problem
 - Adaptive medical treatment, deep research agent, AI scientist, etc.
 - Transition dynamics unknown, reward unknown (and too many states).
 - Interact with the environment to collect data.
 - Great potential for RL, but not realized yet.

2

2

Agenda

- Monte Carlo Policy Evaluation
- Temporal Difference Policy Evaluation
- Model-Free Control

3

3

Model-Free RL

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- So far we assume that we **know all four important sets of information**:
 - Action Space A
 - State Space S
 - Transition Matrix P
 - Reward Function R
- Now, we relax this assumption and assume we **do not know the reward function and transition matrix**.

4

4

Monte-Carlo RL

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- Model-free learning:
 - You do not know reward function and transition matrix.
 - However, you can take actions and **experiment with the environment** to observe the outcomes.
- **Goal**: Given a policy, learn the value function from data.
- Monte-Carlo method:
 - MC RL learns directly from **episodes of experience**. All episode must terminate.
 - MC is model-free, without any knowledge of MDP transitions or rewards.
 - MC leverages the simplest possible idea: **value function = mean return**.

5

5

MC Policy Evaluation

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- Objective: Estimate the value function v_π under policy π :

$$S_0, A_0, R_0, \dots, S_T \sim \pi$$

$$G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$$

$$v_\pi(s) = \mathbb{E}_\pi [G_t \mid S_t = s]$$

- The idea of Monte Carlo: use empirical mean return to approximate expected return.
- **First-visit MC**: v_π is estimated by the average return following each first visit to a state s in a set of episodes.
- **Every-visit MC**: v_π is estimated by the average return following each visit to a state s in a set of episodes.
- If there is an absorbing state, everything is well-defined; otherwise, we need to specify a finite T , which is larger if γ is larger.

6

6

First-Visit MC Policy Evaluation

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- **Initialization:**
 - N (counter), $N(s) \leftarrow 0$ for all $s \in \mathcal{S}$
 - $\text{Returns}(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$
- **Repeat:**
 - Generate** an episode following policy π
 - For each** distinct s appearing in the episode
 - $G \leftarrow$ return following the first occurrence of s
 - $N(s) \leftarrow N(s) + 1$
 - $\text{Returns}(s) \leftarrow \text{Returns}(s) + G$
- **Output:**
 - For each** distinct s
 - $N^{-1}(s)\text{Returns}(s) \longrightarrow v_{\pi}(s)$ as $N(s) \rightarrow +\infty$

- First-visit evaluation is unbiased, and it converges to the true value function.

7

7

Every-Visit MC Policy Evaluation

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- **Initialization:**
 - $N \leftarrow$ counter, $N(s) \leftarrow 0$ for all $s \in \mathcal{S}$
 - $\text{Returns}(s) \leftarrow$ an empty list, for all $s \in \mathcal{S}$
- **Repeat:**
 - Generate** an episode following policy π
 - For each** s appearing in the episode
 - $G \leftarrow$ return following the occurrence of s
 - $N(s) \leftarrow N(s) + 1$
 - $\text{Returns}(s) \leftarrow \text{Returns}(s) + G$
- **Output:**
 - For each** distinct s
 - $N^{-1}(s)\text{Returns}(s) \longrightarrow v_{\pi}(s)$ as $N(s) \rightarrow +\infty$

- Every-visit evaluation is biased, but it still converges to the true value function. And it usually has a smaller MSE because the sample size is much larger.

8

8

Incremental MC Policy Evaluation

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- An incremental way to compute mean value:

$$\begin{aligned}\mu_k &= \frac{1}{k} \sum_{j=1}^k x_j \\ &= \frac{1}{k} \left(x_k + \sum_{j=1}^{k-1} x_j \right) \\ &= \frac{1}{k} (x_k + (k-1)\mu_{k-1}) \\ &= \mu_{k-1} + \frac{1}{k} (x_k - \mu_{k-1})\end{aligned}$$

- Similarly, for MC policy evaluation:

$$\begin{aligned}N(S_t) &\leftarrow N(S_t) + 1 \\ V(S_t) &\leftarrow V(S_t) + \frac{1}{N(S_t)} (G_t - V(S_t))\end{aligned}$$

9

9

Incremental MC Policy Evaluation

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- This incremental update also means less memory usage.
- More importantly, it allows for non-stationary policy (i.e., the transition matrices may change over time):
 - We only consider stationary policy so far, because a stationary MDP with finite states is guaranteed to have a stationary optimal policy.
 - If we go beyond this setting, i.e., policy at time t is different from policy at time $t + 1$.

$$V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$$

10

10

Blackjack with MC

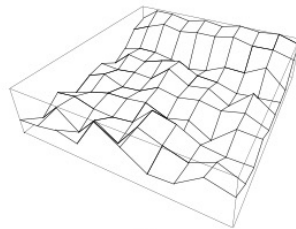
<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- Policy: **Stick** if sum of cards ≥ 20 , otherwise **twist**.

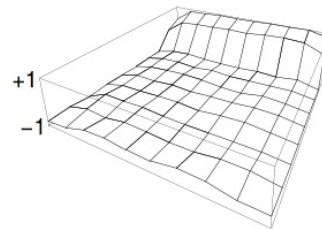
After 10,000 episodes

After 500,000 episodes

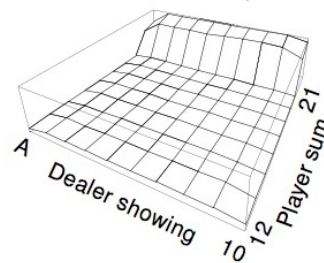
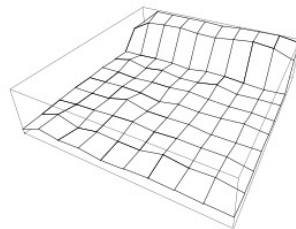
Usable
ace



+1
-1



No
usable
ace



11

11

Agenda

- Monte Carlo Policy Evaluation
- Temporal Difference Policy Evaluation
- Model-Free Control

12

12

Temporal-Difference Learning

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- "If one had to identify one idea as central and novel to RL, it would undoubtedly be temporal-difference (TD) learning." - Sutton and Barto (2017)
- TD is a combination of MC and Bellman operator.
- TD is a model-free method.
- TD can be used in episodic or non-episodic settings.
- TD updates a guess towards a guess.

13

13

Temporal-Difference Learning

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- Recall that, given a policy π , the value function can be obtained as:

$$G_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \gamma^3 r_{t+3} + \dots$$
 in MDP M under policy π

$$V^\pi(s) = \mathbb{E}_\pi[G_t | s_t = s]$$

- Bellman operator and value iteration:

$$B^\pi V(s) = r(s, \pi(s)) + \gamma \sum_{s' \in S} p(s'|s, \pi(s)) V(s')$$

- The incremental MC: $V(S_t) \leftarrow V(S_t) + \alpha (G_t - V(S_t))$

■ Simplest temporal-difference learning algorithm: TD(0)

- Update value $V(S_t)$ toward estimated return $R_{t+1} + \gamma V(S_{t+1})$

$$V(S_t) \leftarrow V(S_t) + \alpha (R_{t+1} + \gamma V(S_{t+1}) - V(S_t))$$

TD can be thought of as a bootstrap method to update based on an existing estimate.

- $R_{t+1} + \gamma V(S_{t+1})$ is called the TD target
- $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ is called the TD error

14

14

TD(0) Algorithm

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

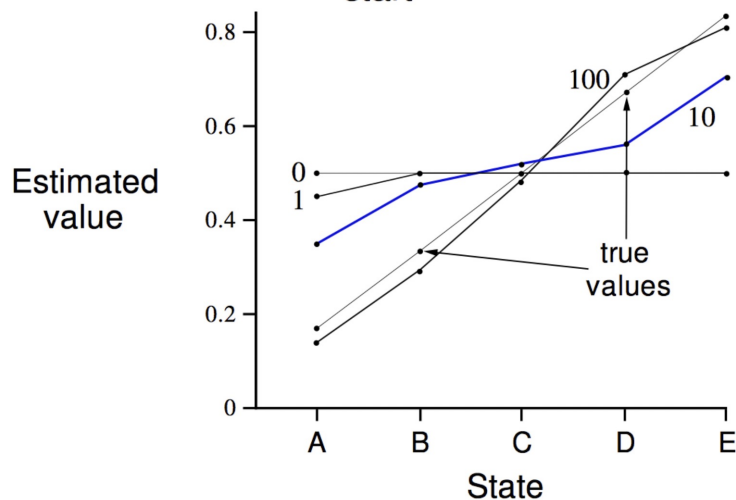
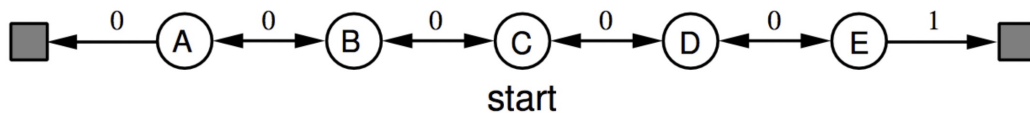
- **Input:** π policy to be evaluated, step size α
- **Initialization:** V arbitrary
- **Repeat** for each episode:
 - Initialize** state s
 - Repeat** for each step of the episode:
 - $a \leftarrow$ action given by π for s
 - Take action a , observe reward r and next state s'
 - $V(s) \leftarrow V(s) + \alpha[r + \gamma V(s') - V(s)]$
 - $s \leftarrow s'$
 - until** s is a terminal state

15

15

Random Walk Example

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>



16

16

MC vs. TD

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

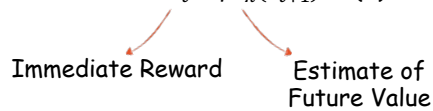
- Recall under the MDP model:

$$V^\pi(s) = \mathbb{E}^\pi(G_t | S_t = s) \quad (1)$$

$$= \mathbb{E}^\pi\left(\sum_{k=0}^{\infty} \gamma^k R_{t+k} | S_t = s\right)$$

$$= \mathbb{E}^\pi[R_t + \gamma V^\pi(S_{t+1}) | S_t = s] \quad (2)$$

- MC methods use G_t in (1) as the target.
- TD methods use $R_t + \gamma v_\pi(S_{t+1})$ in (2) as the target.



17

17

MC vs. TD: Pros and Cons

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- | | |
|---|---|
| • MC must wait until the end of an episode. | • TD can learn online after each step. |
| • MC learns from complete sequences. | • TD can learn from incomplete sequences. |
| • MC only works for episodic environments. | • TD works in continuing environments. |

18

18

Bias-Variance Trade-off

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- Return $G_t = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T$ is *unbiased* estimate of $v_\pi(S_t)$
- True TD target $R_{t+1} + \gamma v_\pi(S_{t+1})$ is *unbiased* estimate of $v_\pi(S_t)$
- TD target $R_{t+1} + \gamma V(S_{t+1})$ is *biased* estimate of $v_\pi(S_t)$
- TD target is much lower variance than the return:
 - Return depends on *many* random actions, transitions, rewards
 - TD target depends on *one* random action, transition, reward

19

19

MC vs. TD: Bias-Variance Trade-off

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

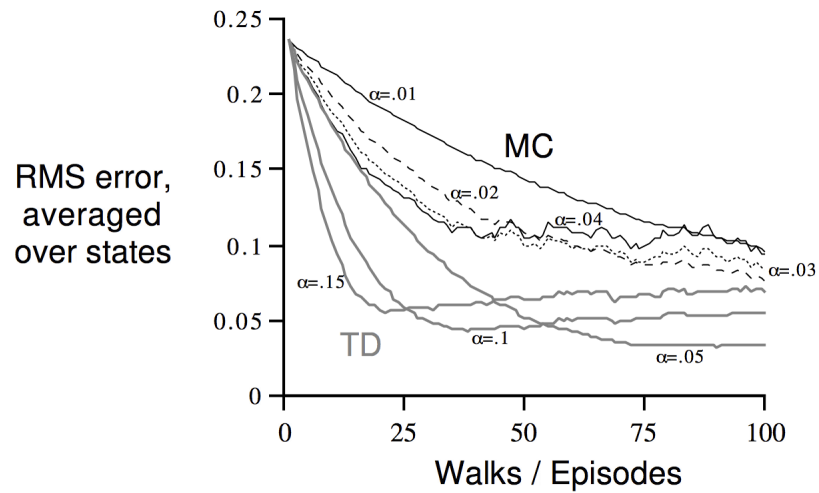
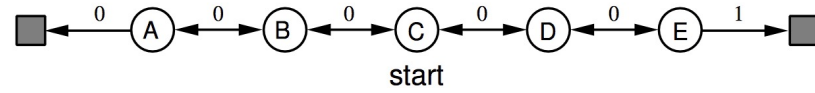
- | | |
|--|---|
| <ul style="list-style-type: none"> • MC updates on G_t after each episode, which is an <i>unbiased estimate</i> of $v_\pi(S_t)$. • MC has high variance, zero bias. <ul style="list-style-type: none"> • Good convergence even with <i>function approximation</i> (using, e.g., DNN). • <i>Not sensitive</i> to initial values. • Simple idea <i>without even knowing Bellman equation</i>. • More efficient in <i>non-Markov environments</i>. | <ul style="list-style-type: none"> • TD updates on $R_t + \gamma v(S_{t+1})$, not $R_t + \gamma v_\pi(S_{t+1})$, which is a <i>biased estimate</i> of $v_\pi(S_t)$. • TD has much lower variance, some bias. <ul style="list-style-type: none"> • Return depends on <i>many</i> random actions/transitions/rewards. • TD target only depends on <i>one</i> random action/transition/reward. • <i>More efficient</i> than MC, because it exploits Markov property and the Bellman equation. • TD(0) converges (but not always with function approximations). • <i>More sensitive to initial values</i>. |
|--|---|

20

20

MC vs. TD: Random Walk Example

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>



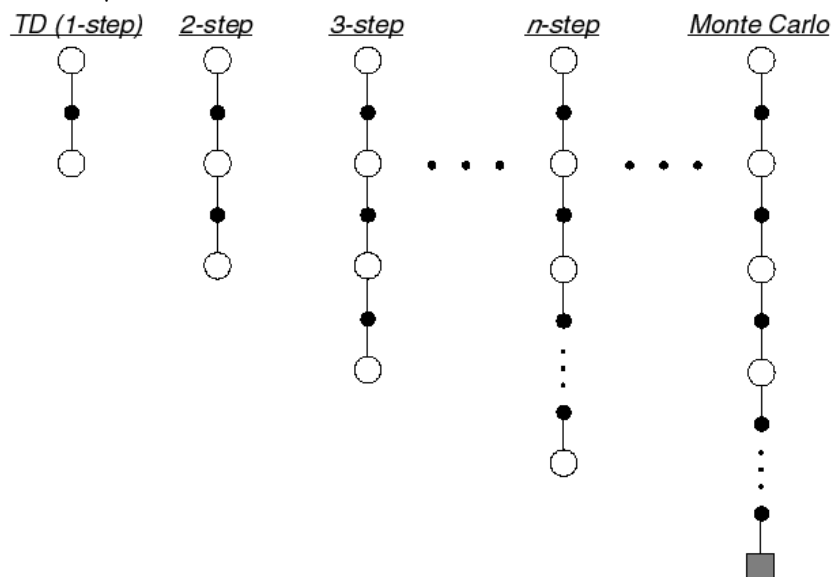
21

21

N-Step Prediction

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- Let TD target look n steps into the future.



22

22

N-Step Return

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>

- We define the n-step returns:

$$\begin{aligned}
 n = 1 \quad (TD) \quad G_t^{(1)} &= R_{t+1} + \gamma V(S_{t+1}) \\
 n = 2 \quad G_t^{(2)} &= R_{t+1} + \gamma R_{t+2} + \gamma^2 V(S_{t+2}) \\
 \vdots \quad &\vdots \\
 n = \infty \quad (MC) \quad G_t^{(\infty)} &= R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T
 \end{aligned}$$

- N-step Temporal-Difference learning, TD(n):

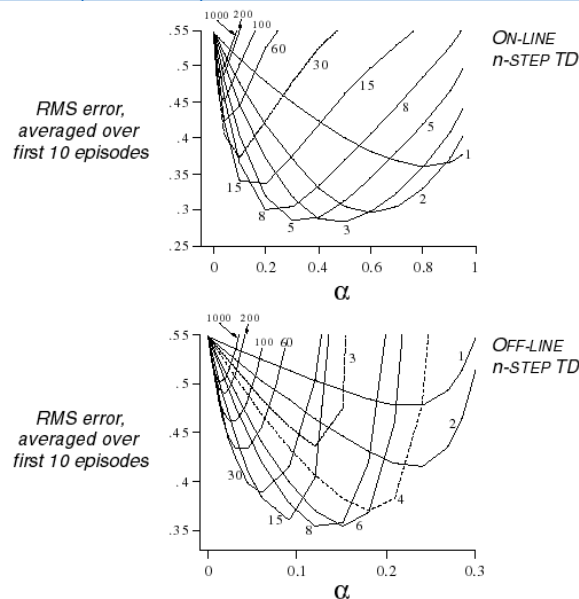
$$V(S_t) \leftarrow V(S_t) + \alpha (G_t^{(n)} - V(S_t))$$

23

23

Random Walk Example with TD(n)

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-4-model-free-prediction-.pdf>



24

24

Agenda

- Monte Carlo Policy Evaluation
- Temporal Difference Policy Evaluation
- **Model-Free Control**

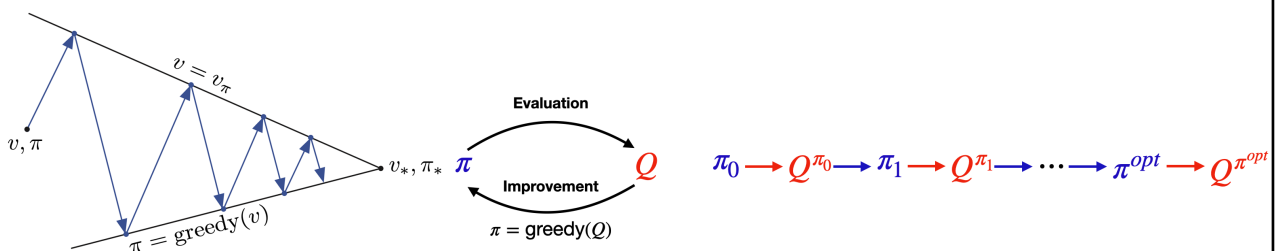
25

25

MC Policy Iteration

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- Objective: use MC estimation to learn the optimal policy.
- MC marries policy iteration:



- Policy Evaluation: Compute v_π via MC policy evaluation.
- Policy Iteration: Improve the current policy w.r.t. the current state-action value function Q_π .
- Challenge: under a policy, many state-action pairs may **never be visited**.
- Solution: **epsilon-greedy**.

26

26

MC Policy Iteration with ϵ -Greedy

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- **Initialization:**

N (counter), $N(s, a) \leftarrow 0$ for all $s \in \mathcal{S}$, $a \in \mathcal{A}$

$\text{Returns}(s, a) \leftarrow$ empty lists, for all $s \in \mathcal{S}$, $a \in \mathcal{A}$

$\pi \leftarrow$ arbitrary ϵ -greedy policy

$Q \leftarrow$ arbitrary

- **Repeat:**

Generate an episode using exploring starts and policy π

For each distinct (s, a) appearing in the episode

$G \leftarrow$ return following the first occurrence of (s, a)

$N(s, a) \leftarrow N(s, a) + 1$

$\text{Returns}(s, a) \leftarrow \text{Returns}(s, a) + G$

$Q(s, a) \leftarrow \text{Returns}(s, a) / N(s, a)$

For each distinct s :

$$\pi(a|s) \leftarrow \begin{cases} \epsilon/m + 1 - \epsilon, & \text{if } a = \arg \max Q(s, a) \\ \epsilon/m, & \text{otherwise} \end{cases}$$

27

27

MC ϵ -Greedy Improves for Each Iteration

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

Theorem

For any ϵ -greedy policy π , the ϵ -greedy policy π' with respect to q_π is an improvement, $v_{\pi'}(s) \geq v_\pi(s)$

$$\begin{aligned} q_\pi(s, \pi'(s)) &= \sum_{a \in \mathcal{A}} \pi'(a|s) q_\pi(s, a) \\ &= \epsilon/m \sum_{a \in \mathcal{A}} q_\pi(s, a) + (1 - \epsilon) \max_{a \in \mathcal{A}} q_\pi(s, a) \\ &\geq \epsilon/m \sum_{a \in \mathcal{A}} q_\pi(s, a) + (1 - \epsilon) \sum_{a \in \mathcal{A}} \frac{\pi(a|s) - \epsilon/m}{1 - \epsilon} q_\pi(s, a) \\ &= \sum_{a \in \mathcal{A}} \pi(a|s) q_\pi(s, a) = v_\pi(s) \end{aligned}$$

Therefore from policy improvement theorem, $v_{\pi'}(s) \geq v_\pi(s)$

- The argument above relies on the value function to be exact. But model-free methods generate value functions **approximately correct**.

28

28

Greedy in the Limit with Infinite Exploration

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

Definition

Greedy in the Limit with Infinite Exploration (GLIE)

- All state-action pairs are explored infinitely many times,

$$\lim_{k \rightarrow \infty} N_k(s, a) = \infty$$

- The policy converges on a greedy policy,

$$\lim_{k \rightarrow \infty} \pi_k(a|s) = \mathbf{1}(a = \operatorname{argmax}_{a' \in \mathcal{A}} Q_k(s, a'))$$

- For ϵ -greedy, it is GLIE if $\epsilon = 1/k$

29

29

GLIE Monte-Carlo Control

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- Sample k th-episode, using policy π : $\{S_1, A_1, R_1, \dots, S_T, A_T, R_T\} \sim \pi$.
- For each S_t and action A_t in the episode:

$$\begin{aligned} N(S_t, A_t) &\leftarrow N(S_t, A_t) + 1 \\ Q(S_t, A_t) &\leftarrow Q(S_t, A_t) + \frac{1}{N(S_t, A_t)} (G_t - Q(S_t, A_t)) \end{aligned}$$

- Improve policy based on new action-value function:

$$\epsilon \leftarrow 1/k$$

$$\pi \leftarrow \epsilon\text{-greedy}(Q)$$

Theorem

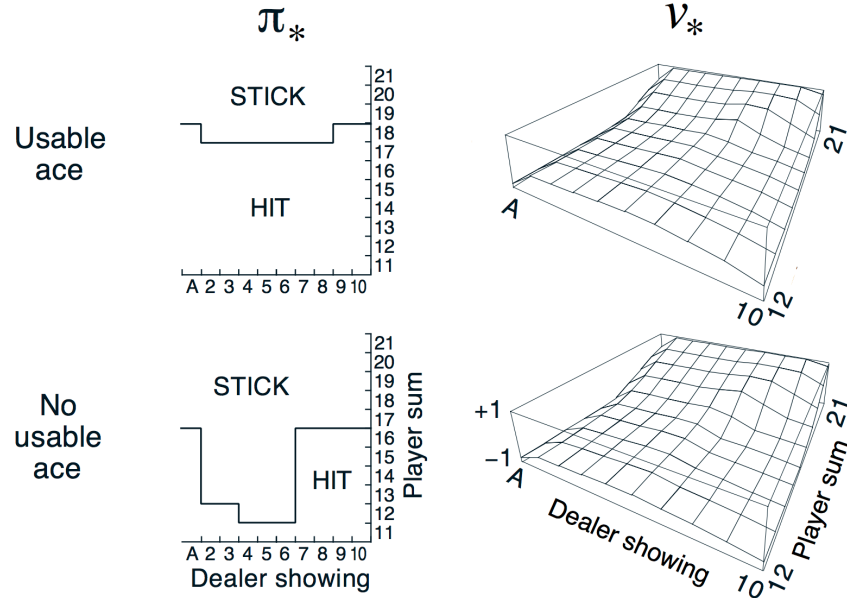
GLIE Monte-Carlo control converges to the optimal action-value function, $Q(s, a) \rightarrow q_(s, a)$*

30

30

Monte-Carlo Control in Blackjack

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>



31

31

Temporal-Difference Control

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- Recall that temporal-difference (TD) has several advantages over Monte Carlo (MC):
 - Low variance
 - Online algorithm
 - Incomplete sequences
- Natural idea: Use TD instead of MC in the control loop:
 - Apply TD to $Q(S, A)$
 - Use ϵ -greedy for policy improvement
- This idea leads to **SARSA** and **Q-learning**.

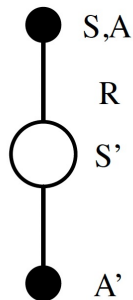
32

32

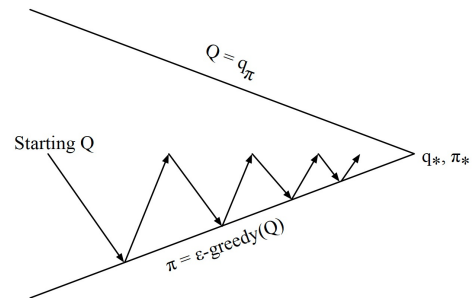
SARSA

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- State $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ State $\rightarrow$ Action (SARSA)
- Update $Q(S, A)$ function using TD with the state and action in the next period $(S', A') \sim \pi$ condition on that the current state and action is (S, A) .



- SARSA is an **on-policy** learning method.



Every **time-step**:

Policy evaluation **Sarsa**, $Q \approx q_\pi$

Policy improvement ϵ -greedy policy improvement

$$Q(S, A) \leftarrow Q(S, A) + \alpha (R + \gamma Q(S', A') - Q(S, A))$$

33

33

SARSA Algorithm

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- 1: Set initial ϵ -greedy policy π , $t = 0$, initial state $s_t = s_0$
- 2: Take $a_t \sim \pi(s_t)$ // Sample action from policy
- 3: Observe (r_t, s_{t+1})
- 4: **loop**
- 5: Take action $a_{t+1} \sim \pi(s_{t+1})$
- 6: Observe (r_{t+1}, s_{t+2})
- 7: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t))$
- 8: $\pi(s_t) = \arg \max_a Q(s_t, a)$ w.prob $1 - \epsilon$, else random
- 9: $t = t + 1$
- 10: **end loop**

34

34

Convergence of SARSA

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

Theorem

Sarsa converges to the optimal action-value function, $Q(s, a) \rightarrow q_*(s, a)$, under the following conditions:

- GLIE sequence of policies $\pi_t(a|s)$
- Robbins-Monro sequence of step-sizes α_t

$$\sum_{t=1}^{\infty} \alpha_t = \infty$$

$$\sum_{t=1}^{\infty} \alpha_t^2 < \infty$$

35

35

N-step SARSA and SARSA(λ)

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- Like TD, we have the n-step SARSA:

$$\begin{array}{ll}
 n = 1 & \text{(Sarsa)} \quad q_t^{(1)} = R_{t+1} + \gamma Q(S_{t+1}) \\
 n = 2 & q_t^{(2)} = R_{t+1} + \gamma R_{t+2} + \gamma^2 Q(S_{t+2}) \\
 \vdots & \vdots \\
 n = \infty & \text{(MC)} \quad q_t^{(\infty)} = R_{t+1} + \gamma R_{t+2} + \dots + \gamma^{T-1} R_T
 \end{array}
 \quad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(q_t^{(n)} - Q(S_t, A_t) \right)$$

- We can also weigh different steps differently using SARSA(λ):

$$q_t^\lambda = (1 - \lambda) \sum_{n=1}^{\infty} \lambda^{n-1} q_t^{(n)}
 \quad Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left(q_t^\lambda - Q(S_t, A_t) \right)$$

36

36

Off-Policy Learning

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- SARSA is on-policy learning, i.e., we learn by playing the policy we are updating.
- Off-policy learning uses another policy to update existing policy.
- We evaluate the **target policy** $\pi(a|s)$ to compute $v_\pi(s)$ and $q_\pi(s, a)$.
- Data is collected from another **behavior policy** $\mu(a|s)$: $\{S_1, A_1, R_1 \dots \dots, S_T, A_T, R_T\} \sim \mu$.
- Why off-policy learning matters?
 - Learn from observing humans or other agents.
 - Re-use experience generated from old policies, $\pi_1, \pi_2, \pi_3, \dots \dots, \pi_{t-1}$.
 - Learn about the **optimal policy** while following **explorative policy**.
 - Learn about **multiple policies** while following one policy.

37

37

Q-Learning

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- Allow both behavior and target policies to improve.
- The target policy π is greedy w.r.t. $Q(s, a)$:

$$\pi(S_{t+1}) = \operatorname{argmax}_{a'} Q(S_{t+1}, a')$$
- The behavior policy μ is ϵ -greedy w.r.t. $Q(s, a)$.
- In Q-learning, we update the target as:

$$\begin{aligned}
 & R_{t+1} + \gamma Q(S_{t+1}, A') \\
 &= R_{t+1} + \gamma Q(S_{t+1}, \operatorname{argmax}_{a'} Q(S_{t+1}, a')) \\
 &= R_{t+1} + \max_{a'} \gamma Q(S_{t+1}, a')
 \end{aligned}$$

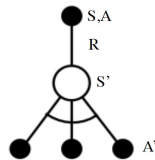
38

38

Q-Learning Algorithm

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- 1: Initialize $Q(s, a), \forall s \in S, a \in A$ $t = 0$, initial state $s_t = s_0$
- 2: Set π_b to be ϵ -greedy w.r.t. Q
- 3: **loop**
- 4: Take $a_t \sim \pi_b(s_t)$ // Sample action from policy
- 5: Observe (r_t, s_{t+1})
- 6: $Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha(r_t + \gamma \max_a Q(s_{t+1}, a) - Q(s_t, a_t))$
- 7: $\pi(s_t) = \arg \max_a Q(s_t, a)$ w.prob $1 - \epsilon$, else random
- 8: $t = t + 1$
- 9: **end loop**



Theorem

Q-learning control converges to the optimal action-value function, $Q(s, a) \rightarrow q_(s, a)$*

$$Q(S, A) \leftarrow Q(S, A) + \alpha \left(R + \gamma \max_{a'} Q(S', a') - Q(S, A) \right)$$

39

39

SARSA vs. Q-Learning

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>

- The difference between SARSA and Q-learning is what they learn.
- For SARSA, we learn the policy the the algorithm plays (on-policy learning):

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_t + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

- For Q-learning, we learn the optimal policy (off-policy learning):

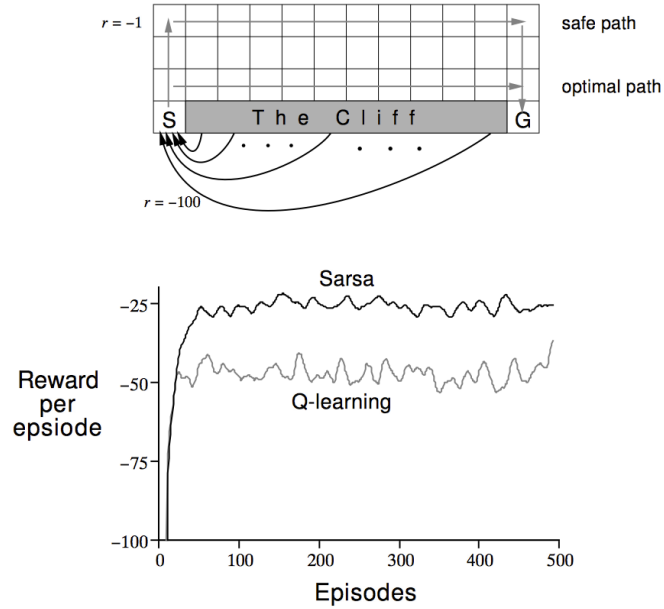
$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha \left[R_t + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t) \right]$$

40

40

SARSA vs. Q-Learning: Cliff Example

<https://davidstarsilver.wordpress.com/wp-content/uploads/2025/04/lecture-5-model-free-control-.pdf>



41